

An Evaluation of Large Language Models for Detection of Malicious Python Packages

Ahmed Ryan¹ Ibrahim Khalil² Abdullah Al Jahid² Md Erfan¹
Sungbin Park³ Akond Ashfaqur Rahman⁴ Md Rayhanur Rahman¹

¹University of Alabama, {aryan9, merfan, mrahman87}@crimson.ua.edu

²University of Dhaka, {bsse1009, bsse1030}@iit.du.ac.bd

³Auburn High School, tp081014@gmail.com

⁴Auburn University, azr0154@auburn.edu

Abstract—Modern software development relies on open-source package repositories where developers share and reuse software components. Attackers increasingly use these repositories to distribute malicious software packages, which embed code intended to harm the host systems that install them. Large Language Models (LLMs) are well-suited to automatic detection of such packages, given their ability to read and reason about code. It remains unclear whether LLMs can go beyond flagging a package as malicious and pinpoint the behaviors that make it so. To this end, we evaluate 13 LLMs on two tasks across a dataset of 4,070 PyPI packages, comprising 370 malicious and 3,700 benign samples. The first task is to detect whether a package is malicious or safe. The second task is to identify malicious indicators (i.e., the specific lines of code that make a package malicious), chosen from a set of 47 known indicator types grouped into 7 categories. We evaluate each LLM across five prompt strategies and three temperatures. For the first task, LLMs show mean F_1 scores ranging from 0.40 to 0.99 across models: they correctly detect most malicious packages but frequently flag safe packages as malicious. For the second task, LLMs show a weighted F_1 score of 0.69 for recognizing the type of malicious behavior but drop to 0.48 when identifying the specific indicators responsible. LLMs correctly identify indicators expressed as recognizable code patterns but miss indicators that require understanding the broader context or the author’s intent. Moreover, LLMs incorrectly report indicators that are not present in the package. We further evaluate the association between indicator identification performance and five factors: model size, context width, prompt strategy, temperature, and code complexity. Of these, only code complexity shows a meaningful association: longer packages are more difficult to analyze. Overall, we advocate that LLMs should be used for initial triage to flag suspicious packages for human review, not as tools that identify what a malicious package does.

Index Terms—Software supply chain security, malicious packages, PyPI, large language models

I. INTRODUCTION

Software packages play a pivotal role in modern software engineering, where developers use package repositories, such as Python Package Index (PyPI) for distributing the packages. While these packages aid in software development, adversaries craft malicious packages, which embed code intended to harm the host systems that install them, and distribute those via the package repositories. For example, on March 31, 2026 adversaries uploaded malicious code in a popular OSS package called ‘litellm’ [1] that harvested cloud credentials and API keys. This Python package was downloaded 3 million

times in one day. The attack was estimated to impact 40,000 downloads [1]. As another example, in September 2025, adversaries injected malicious code to steal cryptocurrency into 18 widely used npm packages, which collectively account for over two billion weekly downloads [2]. According to one research study, the frequency of malicious packages increased by 156% in one year [3].

The above discussion highlights the importance of detecting malicious packages, i.e., software packages in which adversaries intentionally embed harmful functionality to compromise the confidentiality, integrity, or availability of a system. The recurrence of security attacks via malicious software packages has garnered attention and initiatives in regards to securing Open Source Software (OSS) packages and ecosystems from industry, and academics [3], [4]. Despite these efforts, malicious software packages are prevalent. For multiple factors, such as time pressure [5], [6], complexity in resolving third party dependencies [7], and threat sophistication [6], maintainers can miss harmful functionality in source code, which in turn leads to malicious packages being undetected.

We use Figure 1 as an example in this regard. The figure presents a Python code snippet obtained from a package called 11Cent-999.0.4 which is indexed in the pypi_malregistry dataset [8]. The code in *line no 3* is an example of ‘Data Exfiltration’. This code snippet is an indicator, i.e., a code snippet that makes a package malicious. The malicious package has a total of 75 lines of code. If a user is not aware of the package being malicious, then the code snippet will reside in the user’s source code and send sensitive directory data to the adversary.

```
1 ploads = {'/': ld0, '/root/': ld1, '/home/': ld2, '/  
  /root/.ssh/': ld3}  
2 headers = {'content-type': 'application/x-www-form-  
  urlencoded'}  
3 requests.post('https://j0j0.xyz/lists/', data=  
  ploads, verify=True, headers=headers)
```

Listing 1: An example of ‘Data Exfiltration’, which is an indicator of a malicious Python package.

One approach to aid maintainers can be usage of large language models (LLMs) because of their capabilities in secure software development tasks, such as penetration testing, fuzzing, vulnerability discovery, malware analysis, and code

auditing [9]–[12]. Recent works show the lack of automation as a challenge for practitioners when it comes to securing software packages, which makes the usage of LLMs a potential approach to automatically detect malicious packages. A systematic empirical investigation can determine how accurately LLMs can detect malicious software packages in real-world ecosystems, how well they identify indicators, (i.e., coding patterns in Python source code that make a Python package malicious), what are the causes of detection performance, and which factors influence their detection performance.

The goal of this paper is to help practitioners and researchers in understanding the capabilities of LLMs and how they perform in malicious package detection and indicator identification by conducting a systematic empirical evaluation of LLMs’ performance. We investigate the following research questions:

- *RQ1: How accurately can large language models (LLMs) detect malicious Python packages? How accurately can LLMs detect indicators of malicious Python packages?*
- *RQ2: What are the root causes for LLMs’ performance in detecting malicious Python packages and indicators of malicious Python packages?*
- *RQ3: How are parameter size, context width, prompt strategy, temperature, and code complexity related with LLMs’ performance in detecting indicators of malicious packages?*

We conduct an empirical study on 13 Large Language Models (LLMs) based on two tasks i.e. (a) package detection and (b) category recognition and indicator identification to answer our research questions. For RQ_1 , we report precision, recall, and F_1 at three granularities namely, package, category, and indicator. We also compare LLM performance across prompt strategies, temperatures, and LLM categories. For RQ_2 , we partition every response of second task into eight mutually exclusive outcomes, rank the most-missed and most-hallucinated indicators, identify reasons behind outcomes, and group them by investigation effort. For RQ_3 , we measure the association between F_1 and five factors (parameter size, context width, temperature, prompt strategy, and code complexity) using non-parametric statistics (Spearman’s ρ and the Kruskal-Wallis test) interpreted by effect size.

Contributions: We list our contributions as follows:

- An evaluation of 13 LLMs across three granularities: package detection, category recognition, and indicator identification;
- An eight-outcome taxonomy with investigation-effort grouping that exposes a systematic LLMs bias towards under-detecting context-dependent indicators and over-applying labels with recognizable syntax.
- An evaluation of how parameter size, context width, temperature, and prompt strategy are related with indicators of malicious Python packages.

II. KEY CONCEPTS

In this section, we present several key concepts.

Malicious Indicator: An observable signal in package metadata, source code, or runtime behavior that suggests malicious intent. Indicators can be coarse-grained (e.g., suspicious package names), fine-grained (e.g., Application Programming Interface (API) call sequences, sensitive-source to untrusted-sink data flows). Indicators can even span statement and function levels, where individual indicators may be benign in isolation but malicious in combination. In Figure 1, *Data Exfiltration* indicator is present in line number three.

Malicious Indicator Category: A malicious indicator category is a high-level grouping of related malicious indicators based on common behavior. The taxonomy we adopt organizes the 47 indicators into 7 categories [13]. For example, *Typosquatting*, and *Combosquatting* indicators belong to *Metadata Manipulation* category.

Detection, Recognition, and Identification: *Detection* is the verdict of whether a package is malicious or benign. *Recognition* is the assignment of one or more of the seven high-level malicious-indicator *categories* to a package (i.e., which kind of malicious behavior is present). *Identification* is the assignment of one or more of the 47 malicious indicators to a package (i.e., which malicious behavior is present).

Weighted and Macro Average: A weighted average accounts for class frequency, reflecting overall LLMs performance across the dataset. In contrast, a macro average treats all classes equally, highlighting performance on rare classes.

Hallucination: A phenomenon where LLMs generate false, fabricated, or logically inconsistent information while presenting it as factual truth [14].

III. RELATED WORK

In this section, we present the related works.

Malicious Package Detection: Prior work has built datasets of real-world malicious packages across PyPI, Node Package Manager (NPM), and RubyGems [15]–[20]. Recent datasets have added statement-level indicator taxonomies [13] and ATT&CK-aligned labels [21]. Empirical studies characterize malicious-package behavior and the high false-positive rates of existing detectors [8], [16], [19], [20], [22]. Detection techniques comprises metadata and code-metric features [23]–[26], installation-script anomaly detection [27]–[29], source-versus-artifact differential analysis [30]–[33], dataflow, call-graph, slicing, and bytecode analysis [34]–[37], dynamic sandboxing and kernel-level tracing [38]–[43], and traditional, graph, and ensemble learning over these features [44]–[47].

LLM-based Detection: LLMs are increasingly used as classifiers for malicious code, often with few-shot or retrieval-augmented prompting [48], [49]. LLMs are also integrated into static pipelines to flag sensitive APIs, refine features, or guide slicing [50]–[52]. Recent agentic works include multi-agent dissection [53], agent workflows with suspicious-API knowledge and external tools [54], knowledge mining [55], and adaptive registry-to-enterprise detectors [56]. LLMs also summarize malicious behavior [11] and generate YARA/Semgrep rules [12]. MalLoc [57] is the closest to our work. This work localizes malicious payloads with LLMs, but targets

Android and does not compare fine-grained localization against coarse-grained package detection.

Evaluation of LLMs on Security: Comparative studies report variation in LLM performance with frequent false positives and non-determinism [10], [58]–[60]. The studies also find that LLMs analyze isolated logic but fail when required to track data, and logical dependencies across multiple functions, files, or systems [9], [61], [62]. Agent and benchmark studies report gaps in workflow evaluation in real-world environment [60], [63]–[67]. The studies further report that using aggregate scores to evaluate LLMs hide performance flaws [68]. LLMs identify threats but generate high volumes of false positives and require calibration of sensitivity rather than better core detection capabilities [69]. Other studies report that LLM effectiveness is task-dependent and often below expert-designed or hybrid methods [70]–[72]. A study related to our work, evaluates whether LLMs can detect malicious updates to NPM packages [58], but that study does not separate package-level detection from indicator-level identification.

Existing evaluations measure a single granularity (usually a package-level verdict) and rarely test whether LLMs can identify the specific behaviors responsible for that verdict. We evaluate 13 LLMs at both the package and indicator levels over a 47-indicator statement-level taxonomy [13] and analyze the failure patterns behind the gap.

IV. METHODOLOGY

In this section, we present the methodology.

A. Task Formulation

We formulate two classification tasks to evaluate LLMs capabilities for malicious package analysis. **Task 1 (Malicious Package Detection):** A binary classification task designed to determine whether an LLM can distinguish between benign and malicious packages based on source code. **Task 2 (Category Recognition and Indicator Identification):** A multi-label classification task designed to assess whether an LLM can recognize the coarse-grained categories in the code and identify fine-grained indicators.

B. Package Collection

We collect malicious and benign packages to carry out two tasks. Task-1 requires both malicious and benign packages whereas Task-2 requires only malicious packages. With 717,280 total packages in PyPI [73] and over 10,000 recorded malicious packages in *pypi_malregistry* (last accessed June 24, 2026), we estimate the ecosystem’s benign-to-malicious ratio to be $\approx 73 : 1$. Since experimenting with such a ratio is resource-intensive, we maintain a 10:1 benign-to-malicious package ratio for Task-1 following the methodology established in [74]. The resulting ratio remains computationally feasible while ensuring that benign packages outnumber malicious packages to reflect a skewed ecosystem.

Malicious Package Collection: For Task 2, we require malicious Python packages with statement-level labels instead of a single label for the entire package. Statement-level labels

pinpoint the exact Lines of Code (LOC) where malicious behavior is present. To the best of our knowledge, only one study has constructed such a dataset with statement-level labels [13]. The dataset contains **370 malicious packages** (a package comprises one or more files). The packages have a total of 385 files and 2,962 statement-level labels annotated with a taxonomy of 47 indicators. For Task 1, we do not require the statement-level labels, single label for the entire package is sufficient. We collect all the malicious packages of this dataset.

Benign Package Collection: For Task 1, we require benign packages. We initially collect 9,127 benign packages from the dataset provided by the *MalGuard* study [50]. We analyze the dataset and find a heavy-tailed distribution, with a mean of 134,148 LOC but a median of only 288. Malicious packages are usually small, as in our malicious packages, the median LOC is 36. To align with the size of the malicious packages and maintain compatibility with LLMs’ context windows, we filter the benign packages with a minimum of 20 LOC. Then we rank the resulting packages by LOC in ascending order, and choose the top **3,700 benign packages**. In the resulting packages, the highest LOC we observe is 260.

C. Model Selection

We categorize LLMs by availability (open-source vs. proprietary) and specialization (general vs. coder). This setup helps to understand the performance of open-source LLMs against commercial LLMs and whether specialized code training outperforms general reasoning. We select 13 LLMs from 7 different providers to cover variety, as shown in Table I.

Table I: Overview of Selected Large Language Models

Model (Id)	Params (B)	Context (T)	Access	Type
Gemma-3-4B (GEM04)	4	128k	OS	G
Qwen-3-4B (QWEN4)	4	256k	OS	G
Gemma-3-12B (GEM12)	12	128k	OS	G
Phi-4 (PHI04)	15	16k	OS	G
GPT-OSS-20B (GP020)	20	128k	OS	G
Granite-4.0-H-Small (GRANT)	32	131k	OS	G
Devstral Small (DEVST)	24	128k	OS	X
Gemma-3-27B (GEM27)	27	128k	OS	G
Qwen-3 Coder (QWENC)	30	256k	OS	X
Llama-3.3 (LLAMA)	70	128k	OS	G
GPT-OSS-120B (GP120)	120	128k	OS	G
Gemma-4-31B (GEM31)	33	262k	OS	G
GPT-4.1-2025-04-14 (GPT41)	Unknown	1000k	P	G

Note: B=Billions, T=Tokens, OS=Open-Source, P=Proprietary, G=General, X=Coder. Model Ids denote these LLMs throughout the paper.

D. Configuration

Prompt Strategy: Prompt strategy directs the reasoning process. We employ five strategies by increasing complexity and computational cost, to measure how the prompts affect performance. Each successive strategy demands more resources due to larger input contexts, increased token generation, or multiple model iterations. *Zero Shot (ZS)* prompting requires the model to generate a response based solely on pre-trained knowledge without provided examples [75]. *Few-Shot* prompting advances this by including a small number of input-output examples in the context window to demonstrate the desired format or logic [75]. *Chain-of-Thought (CoT)* introduces linear reasoning, prompting the model to output intermediate reasoning steps

before arriving at a final answer [76]. *Self-Consistency* builds upon CoT through parallel consensus, generating multiple independent reasoning paths and selecting the most frequent final answer [77]. *Tree-of-Thoughts (ToT)* utilizes branching logic, enabling the model to explore, evaluate, and backtrack across multiple reasoning steps to solve complex problems [78].

Temperature: We employ three temperature (τ) to measure how determinism and diversity affects performance. At $\tau = 0$, the model deterministically selects the token with highest probability, which can increase repetition [79]. At $\tau = 0.5$ and $\tau = 1$, temperature scaling adjusts output diversity: lower temperature provides coherence, higher temperature provides lexical diversity [79], [80].

E. Sampling Strategy

Our experiment involves 13 LLMs, 5 prompt strategies, and 3 temperatures. We use different sampling strategies for the two tasks, where Task-1 includes both malicious and benign packages, and Task-2 includes only malicious packages.

Task-1 (Binary Classification): We create a 4,070-package corpus (370 malicious + 3,700 benign). A run over such a large corpus would require 793,650 responses (4,070 packages $\times$ 13 models $\times$ 5 prompts $\times$ 3 temperatures), which is computationally expensive. Hence, we sample 2.5% from both 370 malicious and 3,700 benign packages, resulting 10 malicious packages and 93 benign packages. We take one file from each package, and use a fixed seed so that every configuration (prompt, temperature) receives identical packages. We evaluate the open-source model at three temperatures, but we evaluate the proprietary LLM (GPT-4.1) at two temperatures instead of three to reduce expense. In total, we generate 19,570 responses: 103 files $\times$ 12 LLMs $\times$ 5 prompts $\times$ 3 temperatures for open-source and 103 files $\times$ 1 LLM $\times$ 5 prompts $\times$ 2 temperatures for proprietary.

Task-2 (Multi-Label Classification): We use 370 malicious packages (385 files) for every configuration (prompt, temperature). We generate 73,150 responses: 385 files $\times$ 12 LLMs $\times$ 5 prompts $\times$ 3 temperatures for open-source and 385 files $\times$ 1 LLM $\times$ 5 prompts $\times$ 2 temperatures for proprietary.

F. Task Evaluation

We evaluate both tasks using precision, recall, and F_1 score, computed from LLM responses against ground-truth labels.

Task-1 (Binary Classification): We compute precision, recall, and F_1 score across all sampled packages. We aggregate these results to report performance grouped by LLM, prompt strategy, and temperature.

Task-2 (Multi-Label Classification): We evaluate this task at two granularities: the 47 fine-grained indicators and their 7 coarse-grained categories. For category-level evaluation, we map the indicators to their corresponding categories. At both granularities, if a label is present in both the ground truth and the prediction, we count it as a success, regardless of the frequency of the labels. We summarize these per-label scores using macro and weighted averages (defined in Section II).

G. Analysis on Incorrect Indicator Identification

To characterize how the LLMs incorrectly identified indicators (RQ_2), we compare the predicted label set of all LLM responses against the ground-truth label set.

Outcome Taxonomy: We create an outcome taxonomy from the eight possible combinations of the presence (> 0) or absence (0) of True Positive (TP), False Positive (FP), and False Negative (FN) (see Table II). We classify each LLM response into one of eight mutually exclusive outcomes.

Table II: Outcome Taxonomy

Outcome	TP	FN	FP	Status
correct_clean*	0	0	0	Success
correct_hit	> 0	0	0	Success
under_detection	> 0	> 0	0	Partial Success
over_detection	> 0	0	> 0	Partial Success
mixed	> 0	> 0	> 0	Partial Success
false_alarm*	0	0	> 0	Failure
silent_miss	0	> 0	0	Failure
mismatch	0	> 0	> 0	Failure

We interpret Table II. *correct_clean*: LLM correctly identifies a benign package. *correct_hit*: LLM correctly identifies all actual indicators present in the package and does not incorrectly identify any indicator that is not present in the package. *under_detection*: LLM correctly identifies some actual indicators present in the package but does not identify some actual indicators that are present in the package. *over_detection*: LLM correctly identifies all actual indicators present in the package and incorrectly identifies some indicators that are not present in the package. *mixed*: LLM correctly identifies some actual indicators present in the package, does not identify some actual indicators present in the package, and incorrectly identifies some indicators that are not present in the package. *false_alarm*: In a benign package, LLM incorrectly identifies indicator(s) that are not present. *silent_miss*: LLM does not identify any actual indicators present in the package. LLM does not incorrectly identify any indicator that is not present in the package. *mismatch*: LLM does not identify any actual indicator present in the package and incorrectly identifies some indicators that are not present in the package. As every response in Task-2 comes from malicious packages, *correct_clean* and *false_alarm* are zero by construction.

Miss, Hallucination, and Mismatch Analysis: For each indicator, we analyze actual indicators in the package that LLMs do not identify (referred to as miss). We also analyze indicators identified by LLMs that are not present in the package (referred to as hallucination). We rank indicators by these measures to identify the most-missed and most-hallucinated techniques and map each to its corresponding category.

Qualitative and Configuration Analysis: We inspect *mismatch* outcomes of all the 13 LLMs to understand their causes, and we show representative examples for each cause. Finally, we compare the outcome distribution across configurations (prompt strategy, temperature) and report the findings.

Classification based on Investigation Effort: We categorize outcomes by subsequent investigation effort required by human analysts after LLMs triage, ranging from *low* (*over_detection*)

through *medium* (*under_detection*) and *high* (*mixed*) to *critical* (*mismatch*, *silent_miss*), where effort increases with the combination of false positives and missed indicators.

H. Correlation Analysis

To evaluate how different factors influence indicator identification by LLMs, we measure the correlation between the F_1 score and five factors: parameter size, context width, temperature, prompt strategy, and code complexity. For context width, the sample size is the count of LLMs ($N = 13$). For parameter size, we exclude the proprietary LLM GPT41 as we do not know its exact parameter size ($N = 12$). For code complexity, the sample size is the count of malicious packages ($N = 370$). For temperature and prompt strategy, we evaluate all responses ($N = 50,892$) that was successfully parsed. The remaining responses out of the initial 73,150 were excluded due to API timeouts, formatting issues, or execution errors.

As the F_1 score distribution is bounded and non-normal, the analysis relies on non-parametric statistical testing. Spearman’s rank correlation (ρ) evaluates continuous and ordinal factors (parameter size, context width, temperature, and code complexity). The Kruskal-Wallis H test evaluates the nominal prompt strategy variable, deriving the $\eta^2 = H/(N - 1)$ effect size. When the sample size is large ($N \approx 50,000$), a small difference results in a statistically significant p-value, even if the correlation is tiny. To determine if the results are practically significant, we rely on effect sizes, and treat p-value as secondary. We assess correlational strength using absolute rank correlations ($|\rho|$), classifying coefficients < 0.10 as negligible and > 0.50 as large. Concurrently, we quantify the proportion of explained variance using eta-squared (η^2), defining < 0.01 as negligible and > 0.14 as large [81].

V. FINDINGS

In this section, we present the findings.

A. Findings of RQ_1

In this section, we present the findings of RQ_1 .

1) Capability of LLMs in Detection

We present the F_1 scores of LLMs across all configurations for malicious package detection in Table III.

Overall Detection Performance: Across all configurations, mean F_1 scores for the LLMs range from 0.40 to 0.99. We calculate the aggregated mean for each prompt strategy by averaging LLMs’ performance with that prompt across all three temperatures. ZS shows the highest aggregated mean ($F_1 = 0.74$), derived from its mean scores at temperatures 0 ($F_1 = 0.75$), 0.5 ($F_1 = 0.72$), and 1 ($F_1 = 0.76$). ZS is followed by Tree of Thought (ToT) at 0.67, Chain of Thought (CoT) at 0.58, Self Consistency (SC) at 0.58, and Few Shot (FS) at 0.23. Similarly, we calculate aggregated mean for each temperature by averaging LLMs’ performance with that temperature across all five prompts. Temperature 0 produces an aggregated mean of 0.56 (derived from 0.75, 0.25, 0.62, 0.68, and 0.52), while temperature 1 and 0.5 shows 0.58 and 0.53, respectively.

Table III: Package Detection Performance

Prompt	ZS			FS			CoT			ToT			SC			Mean	Max
T	0	0.5	1	0	0.5	1	0	0.5	1	0	0.5	1	0	0.5	1		
GPT41	1	-	1	1	-	1	1	-	1	1	-	1	.95	-	.95	.99	1
LLAMA	.92	.92	.92	.15	.15	.15	.75	.75	.54	.89	.92	.92	.80	.80	.91	.70	.92
QWEN4	.82	.82	.82	.14	.14	.14	.70	.56	.55	.86	1	1	.92	.92	.92	.69	1
GP120	.86	.80	.86	.17	.17	.18	.80	.67	.80	.86	.86	.89	.71	.71	.67	.67	.89
GP020	.83	.83	.77	.18	.21	.17	.29	.29	.36	.92	.92	.91	.80	.83	.83	.61	.92
QWENC	.92	.92	.92	.15	.15	.15	.67	.58	.44	.83	.86	.75	<u>.00</u>	.67	1	.60	1
GEM31	1	1	1	.50	.50	.52	1	1	1	.15	.15	.15	.30	.30	.30	.59	1
PHI04	.15	.14	.15	.15	.14	.15	.55	.52	.39	.82	.75	.75	.86	.86	.92	.49	.92
GEM27	.80	.80	.92	.18	.19	.18	.54	.52	.74	.26	.27	.41	<u>.00</u>	<u>.00</u>	.67	.43	.92
GRANT	.92	.92	.92	.14	.15	.14	.77	.77	.43	.25	.21	.20	.15	.20	.19	.43	.92
DEVST	.29	.27	.32	.16	.15	.14	.14	<u>.21</u>	.19	.86	.75	.80	.67	.63	.80	.43	.86
GEM12	.71	.67	.67	.22	.17	.17	.35	.36	.37	.62	.50	.50	.24	.29	.25	.41	<u>.71</u>
GEM04	.55	.57	.60	.17	.18	.16	.47	.33	.48	.53	.75	.25	.39	.27	.33	<u>.40</u>	.75
Mean	.75	.72	.76	.25	.19	.25	.62	.55	.56	.68	.66	.66	.52	.54	.67		
Best	1	1	1	.50	1	1	1	1	1	1	1	1	.95	.92	1		

Note: Precision (P), Recall (R), F_1 Score (F_1). Combination of Precision and Recall that generated highest F_1 (Best). Zero Shot (ZS), Few Shot (FS), CoT (Chain of Thought), ToT (Tree of Thought), SC (Self Consistency).

The maximum F_1 scores range from 0.71 to 1.00. Four LLMs show maximum score ($F_1 = 1.00$) in at least one configuration. Furthermore, every prompt strategy and temperature setting reaches the maximum score at least once.

Impact of Configuration Selection: An optimal configuration as a combination of prompt strategy and temperature that produces the maximum F_1 score for a given LLM. Evaluated under the optimal configuration, 9 of the 13 LLMs show $F_1 > 0.90$. Four of these LLMs (e.g., GPT41, GEM31, QWEN4, and QWENC) show $F_1 = 1.0$, detecting all malicious packages with zero incorrect predictions. However, the F_1 scores depend on configuration. As mentioned earlier, F_1 scores fluctuated across the 15 configurations, with scores ranging from 0.00 (QWENC at SC, T=0) to 1.0 (QWENC at SC, T=1). As the same LLM can show such difference in F_1 score ($\Delta = 1.0$) based on configuration, reporting an LLM’s F_1 score is meaningless unless the exact configuration is also defined.

Precision and Recall Analysis: In the context of malicious package detection, recall is the proportion of actual malicious packages the LLM detects and precision is the proportion of detected packages that are actual malicious packages. Across all configurations, LLMs maximize recall at the expense of precision. Few actual malicious packages remain undetected by LLMs, 1.6% of configurations result in a recall below 0.5. On the contrary, LLMs produce numerous false positives, 51% of configurations show precision below 0.5. The results demonstrate that most of LLMs incorrect predictions involve detecting benign packages as malicious.

Impact of Prompt Strategy and Temperature: Aggregated mean F_1 scores (defined and calculated in Section V-A1) show that prompt strategy impacts F_1 score more than temperature. Temperatures show a minor variance ≈ 0.05 (ranging from 0.53 to 0.58). Prompt strategy show more variance ≈ 0.51 (ranging from 0.23 to 0.74). The variances indicate that prompt strategy impacts the F_1 score more than temperature by ≈ 10 times.

Limitation of Few-Shot Prompt Strategy: While ZS shows the highest aggregated mean $F_1 = 0.74$, FS shows the lowest aggregated mean $F_1 = 0.23$. For context, if a LLM classifies every package as malicious, it will show F_1 score of 0.14.

Under FS strategy, a single outlier GPT41 shows $F_1 = 1$, but all the other LLMs show F_1 scores ranging from 0.14 to 0.52.

Coder vs General Models: The 2 code-specialized LLMs (QWENC and DEVST) show aggregated mean $F_1 = 0.51$, while the 11 general-purpose LLMs perform slightly better with aggregated mean $F_1 = 0.59$. Both groups reach a maximum $F_1 = 1.0$. For our dataset, specialized code training offers no performance gain in malicious package detection.

2) Capability of LLMs in Recognition and Identification

In this section, we evaluate the performance of LLMs in category recognition and indicator identification.

Overall Recognition and Identification Performance: Table IV shows macro and weighted precision, recall, and F_1 . In category recognition, LLMs show a weighted $F_1 = 0.69$ (Recall 0.63), leaving 37% of categories unrecognized. The macro $F_1 = 0.67$ (Precision 0.79) indicates more false positives in categories with lower support compared to other categories. In indicator identification, LLMs show a weighted $F_1 = 0.48$ (Precision 0.70, Recall 0.41), leaving 59% of indicators unidentified. The macro $F_1 = 0.35$ (Precision 0.43, Recall 0.39) indicates 61% of indicators with lower support remain unidentified.

Table IV: Recognition and Identification Performance

		Mean			Best			
		Average	P	R	F_1	P	R	F_1
Category Recognition	Macro	.79	.65	.67	.79	.71	.71	
	Weighted	.85	.62	.69	.85	.70	.75	
Indicator Identification	Macro	.43	.39	.35	.45	.46	.40	
	Weighted	.70	.41	.48	.69	.47	.52	

Note: Precision (P), Recall (R), F_1 Score (F_1), Combination of Precision and Recall that generated highest F_1 (Best).

Category Recognition Performance: Across six of the seven categories, LLMs show precision scores (0.71 to 0.94) that exceed recall scores (0.38 to 0.85). This result demonstrates that LLMs leave actual categories unrecognized but are correct when they make a prediction. *Network Operations* is the only exception. For this category, recall (0.88) exceeds precision (0.34), indicating that 66% of LLMs predictions in this category are false positives.

Table V: Category Recognition Performance

Category	Support	Mean			Best		
		P	R	F_1	P	R	F_1
Execution Stage (ES)	41,816	.87	.69	.77	.86	.73	.79
Execution Mechanism (EM)	41,431	.94	.85	.89	.93	.92	.92
Defense Evasion (DE)	40,721	.94	.38	.55	.94	.54	.69
Metadata Manipulation (MM)	35,318	.87	.46	.60	.86	<u>.52</u>	.65
System Impact (SI)	15,210	.71	.69	.70	.74	.76	.75
Network Operations (NO)	11,312	<u>.34</u>	.88	<u>.49</u>	<u>.33</u>	.90	<u>.48</u>
Exfiltration (EX)	9,038	.85	.56	.67	.84	.61	.71

Note: Precision (P), Recall (R), F_1 Score (F_1), Combination of Precision and Recall that generated highest F_1 (Best). **Bold** = highest value in column, underline = lowest value in column. Table sorted in descending order by Support.

Mean F_1 score does not depend on sample size. *Execution Mechanism* (Support 41, 431) show a mean F_1 of 0.89. *Defense Evasion* has similar support (40,721) but shows a mean F_1 of 0.55. *Defense Evasion* shows the highest variance

between mean and optimal configurations. Applying optimal configuration increases recall by 0.16 (from 0.38 to 0.54) and the F_1 score by 0.14 (from 0.55 to 0.69) while maintaining a precision of 0.94. *System Impact* has the most balanced precision-recall mean score. The mean precision (0.71) and mean recall (0.69) differ by 0.02. On the contrary, *Defense Evasion* and *Metadata Manipulation* show mean precision-recall differences of 0.56 and 0.41, respectively. When the LLMs predict these categories, the predictions are almost always correct. However, LLMs leave the majority of actual categories unrecognized.

Indicator Identification Performance: We present the identification performance of each indicator in Table VI. For the indicators with highest mean F_1 scores (e.g. *Shell Command Execution* and *Hidden Code Execution*), LLMs are correct in 86% and 92% of their predictions, respectively. Even in these cases, LLMs leave approximately one-third of the actual indicators unidentified, indicating low recall.

Table VI: Indicator Identification Performance

Indicator (Category Codes)	Support	Mean			Best		
		P	R	F_1	P	R	F_1
Suspicious Author Identity (MM)	33223	.90	.40	.55	.88	.43	.58
Shell Command Execution (EM)	30340	.86	.71	.78	.84	.79	.81
Error Suppression (DE)	28908	.85	.19	.31	.85	.28	.42
Encoding-Based Obfuscation (DE)	27906	.89	.30	.45	.87	.39	.53
Hidden Code Execution (EM)	24160	.92	.66	.77	.88	.68	.77
Install-Time Execution (ES)	23668	.51	.52	.51	.51	.61	.56
Decoy Functionality (MM)	23340	.66	.12	.20	.67	.18	.28
Lifecycle Hook Hijack (ES)	15540	.84	.45	.59	.82	.48	.61
System Info Reconnaissance (SI)	12799	.87	.49	.63	.85	.56	.67
Dynamic Evaluation (EM)	8539	.63	.78	.70	.60	.82	.69
Data Exfiltration (EX)	8530	.82	.35	.49	.85	.42	.56
Conditional Payload Trigger (EM)	7742	.70	.23	.34	.66	.29	.40
Typosquatting (MM)	5973	.34	.25	.29	.34	.32	.33
Dynamic Module Import (EM)	5900	.21	<u>.03</u>	<u>.05</u>	.23	<u>.04</u>	.07
Suspicious Connection (NO)	5686	.25	.34	.29	.23	.38	.29
Arbitrary File Write (SI)	5289	.36	.56	.44	.41	.65	.50
Binary Execution (EM)	4432	.40	.35	.37	.37	.41	.39
Encryption-Based Obfuscation (DE)	4385	.93	.45	.60	.91	.60	.72
Suspicious Dependency (MM)	4385	.50	.21	.30	.64	.33	.44
Suspicious Domain Exfiltration (EX)	4208	.31	.16	.21	.33	.16	.22
Payload Dropper (NO)	4021	.16	.49	.24	.16	.59	.25
Description Anomaly (MM)	3531	.29	.59	.39	.34	.68	.45
Computational Obfuscation (DE)	2924	.41	.48	.44	.32	.57	.41
Import-Time Execution (ES)	2774	.10	.20	.14	.11	.25	.16
Binary Dropper (NO)	2748	.25	.83	.38	.25	.83	.38
Combosquatting (MM)	2733	.17	<u>.03</u>	<u>.05</u>	.20	<u>.04</u>	.07
Script File Execution (EM)	1968	.33	.51	.40	.31	.59	.41
File Deletion (SI)	1923	.79	.40	.53	.86	.52	.65
Directory Enumeration (SI)	1823	.31	.54	.40	.27	.56	.36
Dynamic Package Install (EM)	1790	.38	.43	.41	.40	.47	.44
Reverse Shell (NO)	1402	.31	.70	.43	.39	.71	.50
File Relocation (SI)	1280	.78	.35	.49	.90	.38	.54
Embedded String Payload (DE)	966	.12	.07	.09	.10	.10	.10
SSL Validation Bypass (NO)	913	.05	<u>.02</u>	<u>.03</u>	<u>.03</u>	<u>.01</u>	<u>.02</u>
Sensitive Path Write (SI)	830	.18	.34	.23	.18	.41	.25
Environment Modification (SI)	591	.17	.07	.10	.38	.15	.21
Archive Dropper (NO)	571	.75	.56	.64	.80	.70	.75
File Exfiltration (EX)	557	.18	.38	.25	.19	.48	.28
Mining Pool Connection (NO)	285	.73	.68	.70	.88	.75	.81
Webhook Exfiltration (EX)	275	.19	.45	.27	.24	.62	.34
DNS Tunneling (EX)	247	.27	.24	.25	.40	.21	.28
Geolocation Lookup (NO)	216	.10	.20	.13	.13	.29	.18
Script Dropper (NO)	164	<u>.04</u>	.55	.07	.05	.67	.09
ASCII Art Deception (DE)	157	.43	.54	.48	.53	.67	.59
Startup File Persistence (SI)	131	<u>.04</u>	.28	.07	<u>.03</u>	.22	<u>.05</u>
Crypto Wallet Harvesting (SI)	127	.14	.59	.23	.16	.60	.26
Unencrypted Communication (NO)	120	<u>.02</u>	.47	<u>.05</u>	<u>.02</u>	.60	<u>.05</u>

Note: Precision (P), Recall (R), F_1 Score (F_1). Combination of Precision and Recall that generated highest F_1 (Best). **Bold** = top 3 value in column, underline = bottom 3 value in column. Table sorted in descending order by Support.

Multiple indicators show high precision (0.85 to 0.93) but low recall (0.19 to 0.45). For example, *Encryption-Based Obfuscation* shows 0.93 precision and 0.45 recall. Recall drops even further to 0.19 for *Error Suppression*, meaning the model misses 81% of these instances. This trend demonstrates that LLMs avoid false positives at the cost of missing true indicators. *Binary Dropper* and *Reverse Shell* reverse this trend, prioritizing recall over precision. For *Binary Dropper*, LLMs produce 75% false positives but miss only 17% of actual indicators. Similarly, for *Reverse Shell* (precision 0.31, recall 0.70), models generate 69% false positives while missing 30% of indicators. The eight worst-performing indicators show mean $F_1 \leq 0.10$. Despite a large sample size (5,900 instances), *Dynamic Module Import* shows 0.21 precision and 0.03 recall, missing 97% of actual indicators. Performance drops further for *SSL Validation Bypass* (precision 0.05, recall 0.02), where LLMs produce 95% false positives and miss 98% of actual indicators.

Coder vs General Models: The two code-specialized LLMs — DEVST ($F_1 = 0.75$) and QWENC ($F_1 = 0.66$) — show a mean weighted F_1 score of 0.71 for category recognition. In contrast, the eleven general-purpose LLMs show a lower mean weighted F_1 score of 0.64. The general-purpose LLMs show high variance, ranging from a weighted F_1 score of 0.39 for GRANT to 0.79 for GEM31. For indicator identification, the code-specialized LLMs show a mean weighted F_1 score of 0.51, where DEVST evaluates to a weighted F_1 score of 0.56, and QWENC evaluates to 0.46. The general-purpose LLMs show a lower mean weighted F_1 score of 0.41 compared to code-specialized LLMs. The individual scores of general-purpose LLMs range from 0.15 for GRANT to 0.65 for GEM31. The scores show that specialized code training produces higher F_1 score in both category recognition and indicator identification. While performance declines across all LLMs when transitioning from category recognition to indicator identification, code-specialized LLMs retain a higher mean.

Key Takeaway: Under optimal configuration, detection shows mean $F_1 = 0.91$, compared to 0.75 for category recognition and 0.52 for indicator identification, indicating that performance degrades with granularity. Detection is recall-biased, showing more false positives. Identification is precision-biased, leaving more actual indicators unidentified. Prompt strategy improves detection F_1 score upto 0.51 (FS 0.23 to ZS 0.74) but its effect on identification is negligible ($\eta^2 = 0.01$). Coder models underperform general-purpose models in detection ($F_1 = 0.51$ vs. 0.59) but outperform them in identification (0.51 vs. 0.41).

B. Findings of RQ₂

To address RQ₂, we classify all LLM indicator identification responses into a taxonomy based on the presence or absence of TPs, FPs, and FNs (Table VII).

Partial Correctness Over Zero Correct Predictions: The ground-truth set contains the actual indicators present in a package, while the prediction set contains the indicators the LLM identifies. Exact-set matching requires the prediction

Table VII: Indicator Identification Outcomes

Outcome	Count	%
CORRECT_CLEAN	0	0.0%
CORRECT_HIT	135	0.3%
SILENT_MISS	367	0.7%
FALSE_ALARM	0	0.0%
UNDER_DETECTION	8,256	16.2%
OVER_DETECTION	1,092	2.1%
MIXED	37,600	73.9%
MISMATCH	3,442	6.8%
Total Error (Excluding CORRECT_HIT)	50,757	99.7%

set to match the ground-truth set completely. As ground-truth sets often contain eight to nine simultaneous indicators, exact matches occur infrequently. LLMs identify zero correct indicators in 7.5% of responses (*SILENT_MISS* 0.7%, *MISMATCH* 6.8%). On the contrary, LLMs identify at least one correct indicator in 92.2% of responses (*UNDER_DETECTION* 16.2%, *OVER_DETECTION* 2.1%, *MIXED* 73.9%). The scores indicate that LLMs mostly predicts one or more correct indicators, and rarely produces zero correct predictions.

Missed Indicators: Missed indicators are actual indicators present in a package that the LLMs leave unidentified. To analyze these missed indicators, we divide indicators into two categories: recognizable syntax indicators and contextual indicators. Recognizable syntax indicators use standard coding functions, such as executing scripts or modifying files. The code itself provides the evidence, and LLM only needs to identify specific code patterns. Contextual indicators, however, require external knowledge or guessing the author’s true intent.

Table VIII: Top 10 Most-Missed Indicator Techniques

Most-Missed Technique	Tactic	Miss Rate
Combosquatting	Metadata Manipulation	97.2%
Dynamic Module Import	Execution Mechanism	97.0%
Decoy Functionality	Metadata Manipulation	87.9%
Suspicious Domain Exfiltration	Exfiltration	84.5%
Error Suppression	Defense Evasion	81.3%
Suspicious Dependency	Metadata Manipulation	78.7%
Conditional Payload Trigger	Execution Mechanism	77.2%
Typosquatting	Metadata Manipulation	74.9%
Encoding-Based Obfuscation	Defense Evasion	69.8%
Suspicious Author Identity	Metadata Manipulation	60.3%

Mapping the most-missed indicators (see Table VIII) to their corresponding categories shows that most misses cluster within *Metadata Manipulation* and *Defense Evasion*. These categories consist entirely of contextual indicators that require information unavailable to LLMs. Identifying the contextual indicators requires information that is not available to LLMs. For example, identifying *Typosquatting* and *Combosquatting* requires an external popularity index (such as download statistics) to recognize when a malicious package name intentionally mimics a popular legitimate package. Similarly, identifying a *Suspicious Author* requires external threat intelligence to verify if an author account has known malicious history.

Other contextual indicators require subjective analysis of the author’s goals. Identifying *Decoy Functionality* requires such analysis. If the code executes as documented, LLMs cannot distinguish a legitimate feature from a payload. Furthermore,

identifying *Error Suppression* and obfuscation related indicators require contextual judgment to distinguish between poor developer practices, such as broad exception handling, and an attacker deliberately silencing failed malicious executions.

Conversely, miss proportions decrease for recognizable syntax indicators, such as *Install-Time Execution* (miss rate 47.8%), *Arbitrary File Write* (43.7%), *Hidden Code Execution* (34.1%), and *Shell Command Execution* (29.3%). Additionally, the near-total miss proportions for *Dynamic Module Import* (97.0%) and *Combosquatting* (97.2%) indicate that these contextual indicators are rarely identified by LLMs.

Hallucinated Indicators: Hallucination occurs when a LLM incorrectly identifies an indicator that does not actually exist in the code, generating a false positive. Mapping the most-hallucinated indicators (Table IX) show a clustering in specific categories, similar to missed indicators. These false positives occur primarily within *Network Operations* (e.g., payload droppers, external connections) and the *Execution Stage* (e.g., *Install-Time Execution* and *Import-Time Execution*).

Table IX: Most-Hallucinated Indicators

Most-Hallucinated Technique	Tactic	FP Count
Install-Time Execution	Execution Stage	12,109
Payload Dropper	Network Operations	10,554
Binary Dropper	Network Operations	7,004
Suspicious Connection	Network Operations	5,700
Arbitrary File Write	System Impact	5,339
Description Anomaly	Metadata Manipulation	5,099
Import-Time Execution	Execution Stage	5,047
Dynamic Evaluation	Execution Mechanism	3,914

Hallucinations occur because LLMs scan keywords instead of analyzing code. When LLMs encounter common programming functions—such as network requests (`requests.get()`), installation files (`setup.py`), or file writing commands (`open().write()`)—they assume that the code is malicious. As attackers frequently use these recognizable syntax, the LLMs learn to identify those during training. LLMs make this decision based on the presence of the syntax, ignoring the surrounding context and the actual goal of the program. For example, if a LLM encounters a script downloading a safe program, the LLM often misclassifies the action as a *Payload Dropper* because it detects a network command. Similarly, identifying standard setup functions inside a `setup.py` file often causes LLMs to report an *Install-Time Execution* attack. This behavior shows that LLMs fail to trace data flow or understand program intent, relying entirely on matching basic programming patterns to previous examples of malware.

Reasons behind MISMATCH: A *mismatch* occurs when the prediction and ground-truth share zero indicators. Analyzing the responses of 13 LLMs, we identify that *mismatch* outcomes stem from three reasons: ground-truth error, granularity mismatch, and genuine misses. Some representative examples from QWEN4 at ZS and T=0.0 are provided below:

- Ground-Truth Error (Measurement Limitation): The dataset is incorrect or incomplete.
 - `secretinspector-0.1.0`: Ground Truth (GT) = `{Lifecycle Hook Hijack, Reverse Shell}`; Prediction

= `{Shell Command Execution}`. The GT flags a commented-out reverse shell, while the LLM identifies a live shell command (`os.system("echo 'You Have been pwned' > /tmp/pwned")`). The LLM prediction is more accurate than the ground-truth.

- Granularity Mismatch (Capability Limitation): The LLM identifies the correct indicator category but selects another indicator within the same category.
 - `upggrade-requests-0.0.1`: GT = `{..., Combosquatting}`; Prediction = `{Typosquatting}`. LLM recognizes category but selects an adjacent indicator.
 - `cipherbcrypt-1.4`: GT = `{Computational Obfuscation, Data Exfiltration}`; Prediction = `{Encoding-Based Obfuscation, Suspicious Domain Exfiltration, Payload Dropper}`. LLM recognizes obfuscation and exfiltration categories but selects adjacent indicators.
- Genuine Misses (Capability Limitation): The LLM leave the actual indicator unidentified.
 - `j3y5r-0.0.0`: GT = `{Computational Obfuscation, Import-Time Execution, Dynamic Evaluation, Suspicious Author Identity}`; Prediction = `{Typosquatting}`. Execution and obfuscation behaviors stay undetected.

Prompt Strategy Changes Outcome Distribution: A perfect prediction occurs when LLM identifies the exact set of actual indicators in a package. Changing the prompt strategy does not increase perfect predictions, 99.7% of all outcomes remain imperfect. However, different prompts change the distribution of these imperfect outcomes (specifically UNDER_DETECTION, MIXED, and MISMATCH) by shifting LLMs’ willingness to guess versus remain silent.

Table X: Outcome Distribution by Prompt Strategy

Prompt Strategy	UNDER_DETECTION	MIXED	MISMATCH
<code>zero_shot</code>	2,938 (highest)	7,661	811
<code>chain_of_thought</code>	1,756	7,855	626
<code>few_shot</code>	2,110	8,213	1,121 (highest)
<code>tree_of_thought</code>	905	5,877	474
<code>self_consistency</code>	547 (lowest)	7,994	410

The ZS prompt makes the model cautious. It produces the highest rate of *UNDER_DETECTION* outcomes because LLM avoids predicting when uncertain, which leaves actual indicators unidentified. Conversely, the SC prompt triggers over-predicting. It produces the lowest *UNDER_DETECTION* outcomes but pushes most outcomes into the *MIXED* category. In this state, the LLM generates false positives (hallucinations) and false negatives (missed indicators) alongside its correct predictions. The FS prompt shows the highest rate of *MISMATCH* outcomes, producing completely incorrect predictions.

Table XI: Classification based on Investigation Effort

Effort	Associated Outcome	Total (%)
Low	OVER_DETECTION	2.1
Medium	UNDER_DETECTION	16.2
High	MIXED	73.9
Critical	MISMATCH (6.8%), SILENT_MISS (0.7%)	7.5

Classification based on Investigation Effort: We categorize

the outcomes based on subsequent investigation effort required by human analysts after using LLMs for initial triage (Table XI). *OVER_DETECTION* (2.1%) captures all actual indicators but introduces false positives, which requires low effort, as analysts must verify and discard invalid indicators. *UNDER_DETECTION* (16.2%) misses some actual indicators but avoids false positives, which requires medium effort, as analysts must search for the remaining indicators. *MIXED* (73.9%) misses actual indicators and introduces false positives, requires high effort, as analysts must discard invalid indicators and simultaneously search for remaining indicators. *MISMATCH* (6.8%) and *SILENT_MISS* (0.7%) provide zero actual indicators and may add false positives, which requires critical effort, forcing analysts to discard invalid indicators and conduct a full investigation from scratch. Consequently, with over 80% of outcomes requiring high or critical human effort, LLMs do not add much value as triage filters for indicator identification.

Key Takeaway: Responses predominantly show partial correctness: 92.2% contain at least one correct indicator. 73.9% of all responses show *MIXED* outcome. False negatives predominantly occur in contextual categories that require external knowledge, which LLMs do not have. False positives predominantly occur in categories reliant on recognizable syntax, where LLMs predict common code patterns used by adversaries as malicious ignoring surrounding context. Prompt strategy changes the outcome distribution: ZS produces conservative prediction, SC triggers over-prediction, and FS generates the highest proportion of incorrect predictions.

C. Findings of RQ₃

When evaluating data, a measurable difference is not always a meaningful one. Statistical significance, measured by a p -value, indicates whether an observation is reliable or just random noise. p -value less than 0.05 means that the result is due to a genuine correlation rather than random chance. However, a correlation can be statistically significant without being practically useful. To determine if the finding actually matters, we measure the strength of that correlation.

To measure the strength of the correlation, we use statistical tools. We use Spearman’s correlation (ρ) to evaluate the relationship between two variables on a scale from -1 to 1 . A positive value indicates that both variables increase together, a negative value indicates that one increases while the other decreases, and a value near zero indicates no meaningful relationship. For categorical variables, we use Eta-squared (η^2). This metric ranges from 0 to 1 and measures the percentage of the final outcome explained by that variable.

In Table XII, we measure the correlation between indicator-identification F_1 score and five factors: parameter size, context width, prompt strategy, temperature, and code complexity.

Parameter Size: Across the 12 LLMs, increasing the parameter size shows a positive correlation with the F_1 score ($\rho = +0.431$). However, this correlation lacks statistical

significance ($p = 0.16$), meaning the observation could simply be random noise rather than a genuine correlation. Individual LLM results demonstrate this lack of statistical significance. Two LLMs with roughly the same size (32 to 33 billion parameters) achieved the highest (0.65) and lowest (0.17) scores. Furthermore, a smaller 4-billion parameter LLM (0.42) outperformed a larger 15-billion parameter LLM (0.29). As the test included 12 LLMs, the sample size is too small to confirm a reliable correlation. Hence, there is no evidence that a larger model size guarantees better performance.

Table XII: Factors Influencing Indicator-Identification F_1

Factor	Statistic	p	Effect size
Parameter size	$\rho = +0.43$	0.16	Moderate (n.s.)
Context width	$\rho = +0.25$	0.41	Small (n.s.)
Temperature	$\rho = -0.06$	< 0.0001	Negligible
Prompt strategy	$\eta^2 = 0.01$	< 0.0001	Negligible
Code complexity	$\rho = -0.26$	< 0.0001	Small

n.s. = not significant. Prompt strategy uses Kruskal–Wallis ($H = 439.18$); the other factors use Spearman ρ .

Context Width: Across the 13 LLMs evaluated, context width shows a weak positive correlation ($\rho = +0.25$) with the F_1 score. However, this correlation is not statistically significant ($p = 0.41$), suggesting the observation could be random noise rather than a genuine correlation. The analysis is limited by a lack of variation in the data, as most LLMs use a context window of at least 109K tokens. The positive correlation is largely driven by one LLM with a smaller window (16K), which also recorded one of the lowest scores. As most LLMs cluster around similar capacities, there is no dependable evidence in this study that a wider context window improves identification performance.

Temperature: Temperature shows a statistically significant correlation with F_1 scores, meaning the relationship is genuine rather than random noise ($p < 0.0001$). However, the correlation strength is negligible ($\rho = -0.056$), indicating the correlation lacks practical importance. The data shows mean F_1 scores decrease slightly as temperature rises, shifting from 0.470 at $T = 0.0$ to 0.449 at $T = 0.5$ and 0.442 at $T = 1.0$. While lower temperatures lead to more consistent labeling, the shift is too small to matter in practice. Therefore, temperature does not meaningfully impact identification performance.

Prompt Strategy: Prompt strategy has a statistically significant correlation with F_1 scores, indicating a genuine relationship rather than random noise ($p < 0.0001$). However, the effect size is negligible ($\eta^2 = 0.0086$, which falls below the 0.01 threshold for a small effect), meaning the choice of prompt strategy lacks practical importance. Self-consistency (SC) attains the highest mean F_1 (0.486), while zero-shot (ZS) attains the lowest (0.431), representing a difference of approximately 0.055. Other strategies, such as Chain-of-Thought (CoT, 0.463) and Tree-of-Thoughts (ToT, 0.462), do not exceed the performance of self-consistency. These results are specific to the indicator-identification task, the more substantial impact of prompt strategy observed in binary package detection (Section V-A) does not apply here.

Code Complexity: Code complexity has the strongest correlation with F_1 scores, indicating an actual relationship rather than random noise ($p < 0.0001$). Furthermore, the correlation strength is small but meaningful ($\rho = -0.26$), confirming that this factor has the most significant impact on model performance among all factors. Across the 370 packages, a higher LOC is consistently associated with lower F_1 scores. This decline is observed among the largest packages: small (≤ 33 LOC, mean F_1 0.47) and medium (33–34 LOC, 0.49) packages perform similarly, whereas large packages (> 34 LOC, 0.38) show a larger drop. Larger packages contain more indicators and offer more opportunities for performance drop.

Key Takeaway: No LLM setting (parameter size, context width, temperature, or prompt strategy) has a practical effect on indicator-identification: performance cannot be improved by adjusting LLM settings. Only code complexity has significant correlation with indicator identification: performance degrades in larger packages.

VI. DISCUSSION

In this section, we discuss the implications for practitioners and researchers.

A. Implications for Practitioners

LLMs should be used as an initial triage tool to filter malicious packages because models rarely miss a malicious packages (only 1.6% of configurations show < 0.5 recall). High recall makes LLMs well-suited to flag suspicious packages before human review. However, LLMs frequently flag safe packages as malicious (51% of configurations show < 0.5 precision), so a human must confirm every flagged package before any action is taken. LLMs should not be trusted to explain why a package is malicious. At the indicator level, 73.9% of responses are *MIXED*, meaning they contain both correct indicators and false positives, and $> 80\%$ of responses require high or critical analyst effort to clean up. If an analyst trusts an LLM blindly, he will chase fake alerts and miss actual indicators of malicious package.

LLMs should be paired with standard tools to cover blind spots. LLMs miss contextual threats because they lack external data. Instead, analysts should use basic checks: a popularity index for *Typosquatting*, a reputation lookup for *Suspicious Author*, and dataflow analysis to verify if a network call is actually malicious. Larger models, wider context windows, and complex prompts do not improve performance. Therefore, practitioners should deploy small, cost-effective LLMs alongside standard tools rather than relying on massive models. Additionally, standalone F_1 scores are uninformative because performance can fluctuate by $\Delta F_1 = 1.0$. Finally, any reported F_1 score is meaningless without its configuration, since the same LLM can vary by up to $F_1 = 1.0$ across configurations. Therefore, practitioners should always record the prompt strategy and temperature used.

B. Implications for Researchers

LLM performance declines as task becomes more fine-grained: mean $F_1 = 0.91$ for detection under optimal configuration, 0.75 for category recognition, and 0.52 for indicator identification. Detection needs only one indicator to return a correct verdict, recognition needs the category of indicator, identification needs specific indicators, and many indicators need external knowledge such as package popularity, author history, or the author’s intent. As granularity gets finer, the task demands more information, and eventually performance drops. This is a limitation of what LLMs can do, so it motivates supplying the missing information through retrieval over registry, popularity, and authorship data.

Researchers should evaluate identification performance with metrics that reward partial correctness, not exact-set matching. Exact-set matching scores responses with one missed indicator the same as a responses with zero correct indicators, which is misleading because 92.2% of responses contain at least one correct indicator. To evaluate LLMs, researchers should use overlap-based metrics that gives credit to prediction with one or more correct indicators, and category-aware metrics that gives credit to prediction with wrong indicator but right category.

VII. THREATS TO VALIDITY

Construct Validity: Exact-set match penalizes partial correctness, so we also report the full outcome taxonomy and indicator scores. Incomplete or subjective ground-truth labels may artificially lower performance scores. *Internal Validity:* To control for LLM randomness and prompt sensitivity, we used fixed prompt templates. As the test packages are public, models may have seen them during training. While this memorization might artificially inflate detection scores, the models’ failure to identify specific indicators proves it does not grant actual code understanding. *External Validity:* Our study targets PyPI and Python and may not transfer to ecosystems such as NPM or RubyGems. The second task uses only malicious packages, and the benign corpus is restricted to small packages to fit context windows. *Conclusion Validity:* Our small sample size of 12 to 13 models limits statistical power. Consequently, we prioritize effect sizes over p -values and make no causal claims regarding model specifications.

VIII. CONCLUSION

We evaluated 13 LLMs for detecting malicious PyPI packages and identifying their fine-grained indicators across a 47-indicator taxonomy, five prompt strategies, and three temperatures. We observe that the performance of LLMs degrades with granularity. LLMs predominantly produce partially correct responses. LLMs miss context dependent indicators and generates false positives for indicators with recognizable syntax. Prompt strategy changes the outcome distribution. Finally, performance degrades with increasing code complexity. These findings motivate overlap-based and hierarchical evaluation metrics, LLM triage paired with standard tools and information retrieval, fine-tuning and agentic designs to improve indicator identification performance, and extension to other ecosystems.

DECLARATION OF GENERATIVE AI TECHNOLOGIES

During manuscript preparation, the authors used ChatGPT only to improve flow, grammar, and clarity. The tool was not used to generate technical content, synthesize citations, or verify experimental facts. The authors reviewed and verified all outputs and take full responsibility for the final manuscript.

REFERENCES

- [1] Sergio De, "Pypi supply chain attack compromises litellm, enabling the exfiltration of sensitive information," <https://www.infoq.com/news/2026/03/litellm-supply-chain-attack/>, 2026, [Online; accessed 1-April-2026].
- [2] Sygnia. (2025, September) 16 minutes to impact: npm supply chain abuse deploys cryptocurrency-draining malware. Accessed: 2026-03-26. [Online]. Available: <https://www.sygnia.co/threat-reports-and-advisories/npm-supply-chain-attack-september-2025/>
- [3] L. Williams, G. Benedetti, S. Hamer, R. Paramitha, I. Rahman, M. Tamanna, G. Tystahl, N. Zahan, P. Morrison, Y. Acar *et al.*, "Research directions in software supply chain security," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–38, 2025.
- [4] N. Zahan, Y. Acar, M. Cukier, W. Enck, C. Kästner, A. Kapravelos, D. Wermke, and L. Williams, "S3c2 summit 2023-11: Industry secure supply chain summit," *arXiv preprint arXiv:2408.16529*, 2024.
- [5] Linux Foundation Research. (2023, July) Open source maintainers. Accessed: 2026-06-26. [Online]. Available: <https://www.linuxfoundation.org/hubfs/LF%20Research/Open%20Source%20Maintainers%202023%20-%20Report.pdf>
- [6] A. Peruma, T. Choy, G. Lee, and I. D. O. Santos, "Understanding npm developers' practices, challenges, and recommendations for secure package development," *arXiv preprint arXiv:2601.20240*, 2026.
- [7] Chainguard. (2026, February) This shit is hard: The complexities of fixing python library security issues at scale. Accessed: 2026-06-25. [Online]. Available: <https://www.chainguard.dev/unchained/this-shit-is-hard-the-complexities-of-fixing-python-library-security-issues-at-scale>
- [8] W. Guo, Z. Xu, C. Liu, C. Huang, Y. Fang, and Y. Liu, "An empirical study of malicious code in pypi ecosystem," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 166–177.
- [9] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the effectiveness of large language models in detecting security vulnerabilities," in *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 103–114.
- [10] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," IEEE, pp. 862–880, 2024.
- [11] B. Saha, N. Rani, and S. K. Shukla, "Malaware: Automating the comprehension of malicious software behaviours using large language models (llms)," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 2025, pp. 169–173.
- [12] X. Zhang, X. Du, H. Chen, Y. He, W. Niu, and Q. Li, "Automatically generating rules of malicious software packages via large language model," in *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2025, pp. 734–747.
- [13] A. Ryan, J. M. Ifti, M. Erfan, A. A. U. Rahman, and M. R. Rahman, "Unveiling malicious logic: Towards a statement-level taxonomy and dataset for securing python packages," *arXiv preprint arXiv:2512.12559*, 2025.
- [14] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025.
- [15] S. T. Mehedi, R. Jurdak, C. Islam, and G. S. Ramachandran, "Qut-dv25: A dataset for dynamic analysis of next-gen software supply chain attacks," *Advances in Neural Information Processing Systems*, vol. 38, 2026.
- [16] D.-L. Vu, Z. Newman, and J. S. Meyers, "A benchmark comparison of python malware detection approaches," *arXiv preprint arXiv:2209.13288*, 2022.
- [17] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2020, pp. 23–43.
- [18] N. Zahan, P. Burckhardt, M. Lysenko, F. Aboukhadijeh, and L. Williams, "Malwarebench: Malware samples are not enough," in *Proceedings of the 21st international conference on mining software repositories*, 2024, pp. 728–732.
- [19] P. Ladisa, M. Sahin, S. E. Ponta, M. Rosa, M. Martinez, and O. Barais, "The hitchhiker's guide to malicious third-party dependencies," in *Proceedings of the 2023 workshop on software supply chain offensive research and ecosystem defenses*, 2023, pp. 65–74.
- [20] D.-L. Vu, Z. Newman, and J. S. Meyers, "Bad snakes: Understanding and improving python package index malware scanning," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 499–511.
- [21] Y. Zhang, X. Zhou, H. Wen, W. Niu, J. Liu, H. Wang, and Q. Li, "Tactics, techniques, and procedures (ttps) in interpreted malware: A zero-shot generation with large language models," *arXiv preprint arXiv:2407.08532*, 2024.
- [22] X. Zhou, F. Liang, Z. Xie, Y. Lan, W. Niu, J. Liu, H. Wang, and Q. Li, "A large-scale fine-grained analysis of packages in open-source software ecosystems," *arXiv preprint arXiv:2404.11467*, 2024.
- [23] S. Halder, M. Bewong, A. Mahboubi, Y. Jiang, M. R. Islam, M. Z. Islam, R. H. Ip, M. E. Ahmed, G. S. Ramachandran, and M. Ali Babar, "Malicious package detection using metadata information," pp. 1779–1789, 2024.
- [24] J. Yan and B. Zhao, "Pypimaldet: A malicious pypi package detection method combining code features and metadata features," *Neural Networks*, p. 108487, 2025.
- [25] H. Samaana, D. E. Costa, E. Shihab, and A. Abdellatif, "A machine learning-based approach for detecting malicious pypi packages," in *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing*, 2025, pp. 1617–1626.
- [26] P. Ladisa, S. E. Ponta, N. Ronzoni, M. Martinez, and O. Barais, "On the feasibility of cross-language detection of malicious packages in npm and pypi," in *Proceedings of the 39th annual computer security applications conference*, ser. ACSAC '23, 2023, pp. 71–82.
- [27] W. Liang, X. Ling, C. Zhao, J. Wu, T. Luo, and Y. Wu, "Detecting malicious packages in pypi and npm by clustering installation scripts," *IEEE Transactions on Software Engineering*, 2025.
- [28] W. Liang, X. Ling, J. Wu, T. Luo, and Y. Wu, "A needle is an outlier in a haystack: Hunting malicious pypi packages with code clustering," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 307–318.
- [29] G. Liang, X. Zhou, Q. Wang, Y. Du, and C. Huang, "Malicious packages lurking in user-friendly python package index," pp. 606–613, 2021.
- [30] R. Sharifdeen, B. Hassanshahi, M. Mirchev, A. El Hussein, and A. Roychoudhury, "Detecting python malware in the software supply chain with program analysis," in *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2025, pp. 203–214.
- [31] D.-L. Vu, F. Massacci, I. Pashchenko, H. Plate, and A. Sabetta, "Lastpymile: identifying the discrepancy between sources and packages," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 780–792.
- [32] F. N. Froh, M. F. Gobbi, and J. Kinder, "Differential static analysis for detecting malicious updates to open source packages," in *Proceedings of the 2023 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, 2023, pp. 41–49.
- [33] S. Scalco, R. Paramitha, D.-L. Vu, and F. Massacci, "On the feasibility of detecting injections in malicious npm packages," pp. 1–8, 2022.
- [34] Z. Yu, M. Wen, X. Guo, and H. Jin, "Maltracker: A fine-grained npm malware tracker copilot by llm-enhanced dataset," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1759–1771.
- [35] J. Wang, Z. Li, J. Qu, D. Zou, S. Xu, Z. Xu, Z. Wang, and H. Jin, "Malpacdetector: An llm-based malicious npm package detector," *IEEE transactions on information forensics and security*, 2025.
- [36] N. Li, S. Wang, M. Feng, K. Wang, M. Wang, and H. Wang, "Malwukong: Towards fast, accurate, and multilingual detection of malicious code poisoning in oss supply chains," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 1993–2005.
- [37] P. Ladisa, H. Plate, M. Martinez, O. Barais, and S. E. Ponta, "Towards the detection of malicious java packages," pp. 63–72, 2022.

- [38] A. Cohen, "Javasith: A client-side framework for analyzing potentially malicious extensions in browsers, vs code, and npm packages," *arXiv preprint arXiv:2505.21263*, 2025.
- [39] J. Zhang, K. Huang, Y. Huang, B. Chen, R. Wang, C. Wang, and X. Peng, "Killing two birds with one stone: Malicious package detection in npm and pypi using a single model of malicious behavior sequence," *ACM transactions on software engineering and methodology*, vol. 34, no. 4, pp. 1–28, 2025.
- [40] T.-C. Nguyen, D.-L. Vu, and N. C. Debnath, "An analysis of malicious behaviors of open-source packages using dynamic analysis," in *International Conference on Computer Applications in Industry and Engineering*. Springer, 2024, pp. 102–114.
- [41] C. Huang, N. Wang, Z. Wang, S. Sun, L. Li, J. Chen, Q. Zhao, J. Han, Z. Yang, and L. Shi, "{DONAPI}: Malicious {NPM} packages detector using behavior sequence knowledge mapping," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 3765–3782.
- [42] S. T. Mehedi, C. Islam, G. Ramachandran, and R. Jurdak, "Dysec: a machine learning-based dynamic analysis for detecting malicious packages in pypi ecosystem," *IEEE Transactions on Information Forensics and Security*, 2026.
- [43] Z. Tan, K. Xiao, J. Singer, and C. Anagnostopoulos, "Operational runtime behavior mining for open-source supply chain security," *arXiv preprint arXiv:2601.06948*, 2026.
- [44] J. Singh and J. Singh, "Detection of malicious software by analyzing the behavioral artifacts using machine learning algorithms," *Information and Software Technology*, vol. 121, p. 106273, 2020.
- [45] A. Sejfić and M. Schäfer, "Practical automated detection of malicious npm packages," pp. 1681–1692, 2022.
- [46] Y. Huang, W. Zheng, S. Wu, B. Chen, Y. Lu, Z. Zhou, Y. Cao, X. Li, and X. Peng, "Profmal: Detecting malicious npm packages by the synergy between static and dynamic analysis," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025, pp. 419–431.
- [47] T. Iqbal, G. Wu, Z. Iqbal, M. B. Mahmood, A. Shafique, and W. Guo, "PyPIguard: A novel meta-learning approach for enhanced malicious package detection in pypi through static-dynamic feature fusion," *Journal of Information Security and Applications*, vol. 90, p. 104032, 2025.
- [48] M. O. Ibiyo, T. Louangdy, P. T. Nguyen, C. Di Sipio, and D. Di Ruscio, "Detecting malicious source code in pypi packages with llms: Does rag come in handy," in *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, 2025, pp. 653–658.
- [49] X. Sun, X. Gao, S. Cao, L. Bo, X. Wu, and K. Huang, "1+ 1> 2: Integrating deep code behaviors with metadata features for malicious pypi package detection," in *Proceedings of the 39th IEEE/ACM international conference on automated software engineering*, 2024, pp. 1159–1170.
- [50] X. Gao, X. Sun, S. Cao, K. Huang, D. Wu, X. Liu, X. Lin, and Y. Xiang, "{MalGuard}: Towards {Real-Time}, accurate, and actionable detection of malicious packages in {PyPI} ecosystem," pp. 4741–4758, 2025.
- [51] Y. Huang, R. Wang, W. Zheng, Z. Zhou, S. Wu, S. Ke, B. Chen, S. Gao, and X. Peng, "Spiderscan: Practical detection of malicious npm packages based on graph-based behavior modeling and matching," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1146–1158.
- [52] D.-K. Nguyen, G.-T. Ho, Q.-M. Pham, T. A. Dang-Thi, M.-K. Vu, T.-C. Nguyen, P. T. Tran-Truong, and D.-L. Vu, "Taint-based code slicing for llms-based malicious npm package detection," *arXiv preprint arXiv:2512.12313*, 2025.
- [53] T. Toda and T. Mori, "Chase: Llm agents for dissecting malicious pypi packages," in *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware)*. IEEE, 2025, pp. 01–10.
- [54] S. Pang, Z. Jiang, Y. Yao, Z. Fan, H. Li, and B. Liu, "PyPiline: Malicious pypi package detection via suspicious api knowledge and agent workflow," *arXiv preprint arXiv:2606.19063*, 2026.
- [55] W. Guo, C. Liu, M. Kang, Y. Zhang, J. Wu, Z. Xu, V. Sachidananda, and Y. Liu, "Cutting the gordian knot: Detecting malicious pypi packages via a knowledge-mining framework," *arXiv preprint arXiv:2601.16463*, 2026.
- [56] B. Montaruli, L. Compagna, S. E. Ponta, and D. Balzarotti, "One detector fits all: Robust and adaptive detection of malicious packages from pypi to enterprises," *arXiv preprint arXiv:2512.04338*, 2025.
- [57] T. Sun, M. Alecci, A. Pilgun, Y. Song, X. Tang, J. Samhi, T. F. Bissyandé, and J. Klein, "Malloc: Toward fine-grained android malicious payload localization via llms," in *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2025, pp. 761–766.
- [58] E. Wyss, D. Tassio, L. De Carli, and D. Davidson, "Evaluating llm-based detection of malicious package updates in npm," in *2025 28th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. IEEE, 2025, pp. 678–692.
- [59] S.-C. Dai, J. Xu, and G. Tao, "Rethinking the evaluation of secure code generation," *arXiv preprint arXiv:2503.15554*, 2025.
- [60] J. Han, Z. Yu, L. Bao, J. Liu, Y. Wan, J. Yin, S. Deng, and S. Han, "From llms to agents: A comparative evaluation of llms and llm-based agents in security patch detection," *arXiv preprint arXiv:2511.08060*, 2025.
- [61] J. Lin, D. Mohaisen *et al.*, "From large to mammoth: A comparative evaluation of large language models in vulnerability detection," in *NDSS*, 2025.
- [62] L. Huynh, Y. Zhang, D. Jayasundera, W. Jeon, H. Kim, T. Bi, and J. B. Hong, "Detecting code vulnerabilities using llms," pp. 401–414, 2025.
- [63] H. Lee, Z. Zhang, H. Lu, and L. Zhang, "Sec-bench: Automated benchmarking of llm agents on real-world software security tasks," *Advances in Neural Information Processing Systems*, vol. 38, pp. 116 342–116 378, 2026.
- [64] A. Yildiz, S. G. Teo, Y. Lou, Y. Feng, C. Wang, and D. M. Divakaran, "Benchmarking llms and llm-based agents in practical vulnerability detection for code repositories," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 30 848–30 865.
- [65] R. Wang, J. Guo, C. Gao, G. Fan, C. Y. Chong, and X. Xia, "Can llms replace human evaluators? an empirical study of llm-as-a-judge in software engineering," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 1955–1977, 2025.
- [66] J. Feiglin and G. Dar, "Sastbench: A benchmark for testing agentic sast triage," *arXiv preprint arXiv:2601.02941*, 2026.
- [67] M. Damie, M. B. Ertan, D. Essoussi, A. Makhanu, G. Peter, and R. Wensveen, "Tosss: a cve-based software security benchmark for large language models," *arXiv preprint arXiv:2603.10969*, 2026.
- [68] S. Halder, S. Saxena, K. K. Shrish *et al.*, "Seclens: Role-specific evaluation of llm's for security vulnerability detection," *arXiv preprint arXiv:2604.01637*, 2026.
- [69] J. Barnes, "Opensec: Measuring incident response agent calibration under adversarial evidence," *arXiv preprint arXiv:2601.21083*, 2026.
- [70] Y. Zhang, W. Song, Z. Ji, N. Meng *et al.*, "How well does llm generate security tests?" *arXiv preprint arXiv:2310.00710*, 2023.
- [71] S. A. Licorish, A. Bajpai, C. Arora, F. Wang, and K. Tantithamthavorn, "Comparing human and llm generated code: The jury is still out!" *arXiv preprint arXiv:2501.16857*, 2025.
- [72] C. Rondanini, B. Carminati, E. Ferrari, A. Kundu, and A. Gaudiano, "Malware detection at the edge with lightweight llms: a performance evaluation," *ACM Transactions on Internet Technology*, vol. 26, no. 1, pp. 1–24, 2026.
- [73] Pypi download stats. <https://pypistats.org/>. Accessed: 2026-06-01.
- [74] Y. Qing, Q. Yin, X. Deng, Y. Chen, Z. Liu, K. Sun, K. Xu, J. Zhang, and Q. Li, "Low-quality training data only? a robust framework for detecting encrypted malicious network traffic," 2023.
- [75] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [76] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.
- [77] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations (ICLR)*, 2022.
- [78] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," vol. 36, 2023, pp. 11 809–11 822.
- [79] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," *arXiv preprint arXiv:1904.09751*, 2019.
- [80] D. H. Ackley, G. E. Hinton, and T. J. Sejnowski, "A learning algorithm for boltzmann machines," *Cognitive science*, vol. 9, no. 1, pp. 147–169, 1985.
- [81] J. Cohen, *Statistical power analysis for the behavioral sciences*. routledge, 2013.